NG-CONSULTING

LE PROTOCOLE TLS/SSL

Introduction

Comme la plupart des consultants juniors j'ai été véritablement confronté à la problématique du SSL lorsqu'il y a 4 ans. J'essayais de mettre mon premier site en ligne à ma sortie d'école. Il s'agissait d'un site qui devait présenter mon CV aux potentiels employeurs. Je n'ai reçu aucune offre pendant presque un mois. C'est finalement lorsque j'ai pu décocher un premier entretient que la RH m'expliqua très gentiment que tous les navigateurs sur lesquels elle consultait mon CV lui indiquaient que celui-ci était dangereux et de ne surtout pas l'ouvrir. En cherchant un peu, je suis tombé sur la notion SSL. Vous vous doutez bien qu'elle m'a eu au rabais :). Vous trouverez l'article associé ici https://toungafranck.com/le-protocole-tls-ssl/

Table des matières

Chapitre 1. Présentation du protocole TLS/SSL

Chapitre 2. Mécanisme de chiffrement SSL

Chapitre 3. Les certificats

Annexe A. La comparaison entre SSL et TLS

Annexe B. Exemple d’appel REST en java d’une API en https

Chapitre 1 – PRESENTATION DU PROTOCOLE SSL/TLS

Dans cette série d'articles sur la sécurité TLS, nous allons nous concentrer sur deux protocoles largement connus et utilisés dans la sécurité informatique : le SSL et TLS. Nous commencerons par SSL, qui signifie Secure Socket Layer, puis nous parlerons de son successeur, TLS, qui signifie Transport Layer Security.

Beaucoup de gens, utilisent souvent ces termes, mais ne comprennent pas vraiment la technologie sous-jacente et la mécanique derrière ces protocoles. La raison est simple. Derrière ces protocoles, il existe un réseau complexe de fonctions cryptographiques, d'algorithmes et de structures.

Après avoir compris que le fonctionnement de SSL / TLS peut être un défi, j'essaierai d'en donner un aperçu simple mais relativement complet.

Qu'est-ce que TLS / SSL?

L'objectif principal des protocoles TLS (Transport Layer Security) et SSL (Secure Socket Layer) est de fournir : la confidentialité et l'intégrité aussi bien des données échangées que de l'indentification du serveur.

Confidentialité et intégrité

Il permet de crypter la connexion entre deux supports (client-serveur). En chiffrant la communication, nous nous assurons qu'aucun tiers n'est capable de lire ou de falsifier les données échangées sur notre connexion avec le serveur. Une communication non cryptée

exposera des données sensibles telles que les noms d'utilisateur, les mots de passe, les numéros de cartes de crédit et, en général, tout ce qui est renvoyé pendant la connexion.

En utilisant une connexion normale – non cryptée, si un tiers interceptait notre connexion avec le serveur, il serait capable de voir les informations échangées en clair (format lisible par l'homme). Si, par exemple, nous accédons au panneau d'administration de notre site Web sans un protocole SSL et que quelqu'un renifle le trafic du réseau local, ils pourraient voir ce qui suit :

Dans ce cas, l'information est inutile pour le hacker.

Identification

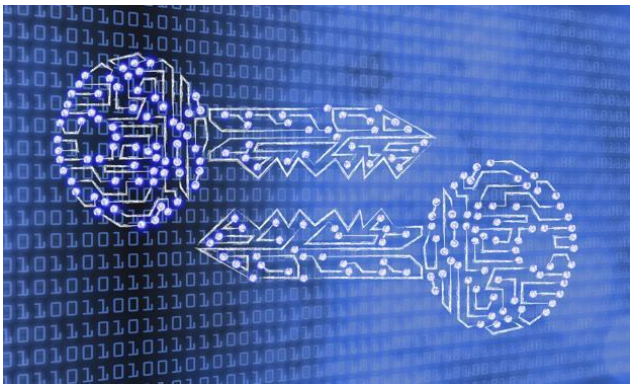
Avec l'utilisation de la cryptographie à clé publique, SSL / TLS fournit une identification entre les parties communicantes. Cela signifie que l'un (le plus souvent le serveur), ou les deux parties, savent avec qui ils communiquent. Ceci est crucial, en particulier dans le cas de transactions en ligne, car nous devons nous assurer que nous transférons de l'argent à la personne ou l'entreprise qui sont ce qu'ils prétendent être.

Lorsqu'une connexion sécurisée est établie, le serveur envoie son certificat SSL au client. Le certificat sera ensuite vérifié par le client auprès d'une autorité de certification de confiance, validant essentiellement l'identité du serveur. Nous verrons comment un certificat SSL est créé plus tard dans cette série.

Application

Les protocoles SSL / TLS peuvent être utilisés dans différents services tels que le Web, le courrier, FTP, VoIP et VPN. Généralement, lorsqu'un service utilise une connexion sécurisée, la lettre S est ajoutée au nom du protocole du service. Par exemple : HTTPS, SMTPS, FTPS, SIPS.

Chapitre 2 : LES MECHANISME DE CHIFFREMENT SSL



Dans la première partie de cette série, nous avons présenté le protocole TLS / SSL ? Dans cette partie de la série, nous décrirons certaines des terminologies TLS / SSL.

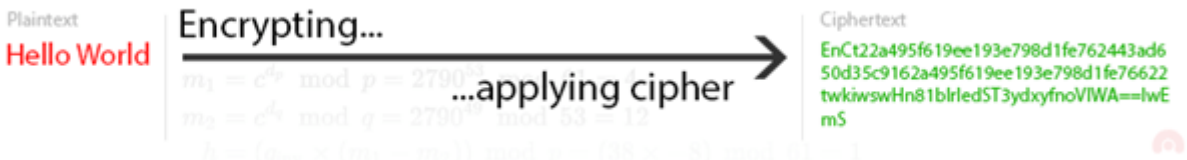
Avant de plonger plus profondément dans le protocole, jetons d'abord un coup d'œil sur les bases. Comprendre ce qui suit vous aidera à mieux comprendre les sujets discutés et analysés plus tard.

Chiffrement

La cryptographie est le processus dans lequel un message lisible par l'homme (texte en clair) est converti en un format crypté, non lisible par l'homme (texte chiffré). L'objectif principal du cryptage est de s'assurer que seul un récepteur autorisé pourra déchiffrer et lire le message original. Lorsque des données non cryptées sont échangées entre deux parties, sur un support quelconque, une tierce partie peut intercepter et lire la communication échangée.

Si l'échange contient des informations sensibles, cela implique une perte de confidentialité. En outre, si le tiers peut intercepter et lire les messages, il peut également altérer les données, ce qui signifie qu'il peut modifier les informations échangées, compromettant ainsi l'intégrité du message.

Imaginez envoyer un paiement sur un canal non crypté. Le paiement comprend vos coordonnées bancaires ainsi que le montant que vous avez autorisé. Un hacker pourrait utiliser une attaque man-in-the-middle pour altérer l'information et changer le montant de 100 $ à 10 000 $. La banque reçoit les données altérées du tiers au lieu de vous, ce qui signifie qu'il n'y a pas d'authenticité. En utilisant le cryptage, un hacker peut toujours être capable d'intercepter le trafic, mais il ne pourra pas lire ou altérer les données.



Chiffrement symétrique

Le cryptage symétrique est le processus dans lequel la même clé est utilisée pour le cryptage et le décryptage des données.

Si Thomas veut envoyer des informations à Bob, il utilisera une clé partagée pour crypter les données et Bob les déchiffrera en utilisant la même clé.

Le plus gros problème avec le chiffrement à clé symétrique est que les données échangées doivent être chiffrées et décryptées avec la même clé. Cela signifie que toutes les parties qui échangent des données doivent avoir la clé partagée.

L'inconvénient majeur de ceci est que si la clé partagée est exposée, un hacker serait capable de déchiffrer toute la communication cryptée avec cette clé. C'est pourquoi la répartition des clés partagées entre les parties doit être effectuée sur un canal de communication crypté sécurisé déjà établi. Un autre inconvénient est que vous ne pouvez pas authentifier l'expéditeur d'un message, ce qui compromet l'authenticité.

Avantages du chiffrement symétrique

- Rapide, faible utilisation des ressources.
- Opération simple.
- Fiable.

Inconvénients du chiffrement symétrique

- Même clé utilisée pour le chiffrement / décryptage.
- La distribution des clés doit être effectuée sur un canal sécurisé déjà établi.
- Une clé différente est nécessaire pour différentes parties – gestion / distribution des clés.
- Impossible d'authentifier les utilisateurs.

Chiffrement asymétrique

Contrairement au cryptage à clé symétrique, le cryptage asymétrique (également appelé cryptographie à clé publique) utilise une paire de clés, une clé publique et une clé privée. Ces clés cryptographiques sont uniquement liées, ce qui signifie que tout ce qui est chiffré avec une clé peut être déchiffré avec l'autre. La clé publique, comme son nom l'indique, peut être partagée avec n'importe qui. La clé privée doit être connue uniquement du serveur.

Le cryptage asymétrique peut être utilisé pour l'authentification de l'expéditeur. Si Bob signe et crypte un message à l'aide de sa clé privée, quiconque le décrypte avec la clé publique de Bob peut être certain que Bob est l'expéditeur.

C'est pourquoi garder une clé privée sécurisée est essentiel.

Avantages du chiffrement asymétrique

La distribution des clés est facile.
Authenticité.
Intégrité.
Garantir.

Inconvénients du chiffrement asymétrique

Cryptage plus lent que celui symétrique.
Besoin de plus de ressources.

Les Ciphers (ou techniques de chiffrements)

Les Ciphers sont des méthodes / algorithmes utilisés pour chiffrer et déchiffrer des données.

Chiffrement par bloc

Dans ce procédé, les données sont divisées en blocs de longueur fixe, puis cryptées (par exemple des blocs de 64 bits ou de 128 bits). Si le dernier bloc des données est inférieur à la longueur de bloc spécifiée, le remplissage sera utilisé pour remplir l'espace « vide ». Les chiffrements de blocs populaires incluent AES, Blowfish, 3DES, DES et RC5.

Le Padding

Les chiffrements par bloc ont une longueur fixe spécifiée et la plupart d'entre eux exigent que les données d'entrée soient un multiple de leur taille. Il est courant que le dernier bloc contienne des données qui ne répondent pas à cette exigence. Dans ce cas, un remplissage (généralement des données aléatoires) est utilisé pour l'amener à la longueur de bloc requise.

Vecteur d'initialisation (IV)

Un vecteur d'initialisation est une entrée de taille fixe aléatoire (ou pseudo-aléatoire) utilisée dans les méthodes de cryptage. Si cette entrée n'est pas répétitive sur chaque message, alors elle est aussi appelée nonce, ce qui signifie qu'elle ne peut être utilisée qu'une seule fois.

Le but principal d'un IV est de démarrer une méthode de cryptage. Dans les modes Cipher, comme CBC (Cipher Block Chaining), où chaque bloc est XORed avec le bloc précédent, dans le premier bloc il n'y a pas de bloc précédent à XOR avec, donc un vecteur d'initialisation est utilisé comme entrée pour le premier bloc pour commencer hors du processus.

Un nonce est également utilisé pour empêcher les hacker de déchiffrer tous les messages en devinant l'IV. Un nonce, qui devrait être aléatoire et imprévisible, permet au même message d'être chiffré avec la même clé et pourtant avoir un résultat différent (texte chiffré).

XOR (Exclusif Ou)

XOR est une fonction logique facile à implémenter qui est utilisée en cryptographie (parmi ses nombreuses autres utilisations). XOR prend des motifs à deux bits et il ne retourne vrai que si les deux entrées sont différentes.

INPUT 1	INPUT 2	OUTPUT
0	0	0
1	0	1
0	1	1
1	1	0

Voici un exemple d'opération de chiffrement XOR.

Message	Hello!	01001000 01000101 01001100 01001100 01001111 00100001
Key	S3CR3T	01010011 00110011 01000011 01010010 00110011 01010100
Ciphertext		00011011 01110110 00001111 00011110 01111100 01110101

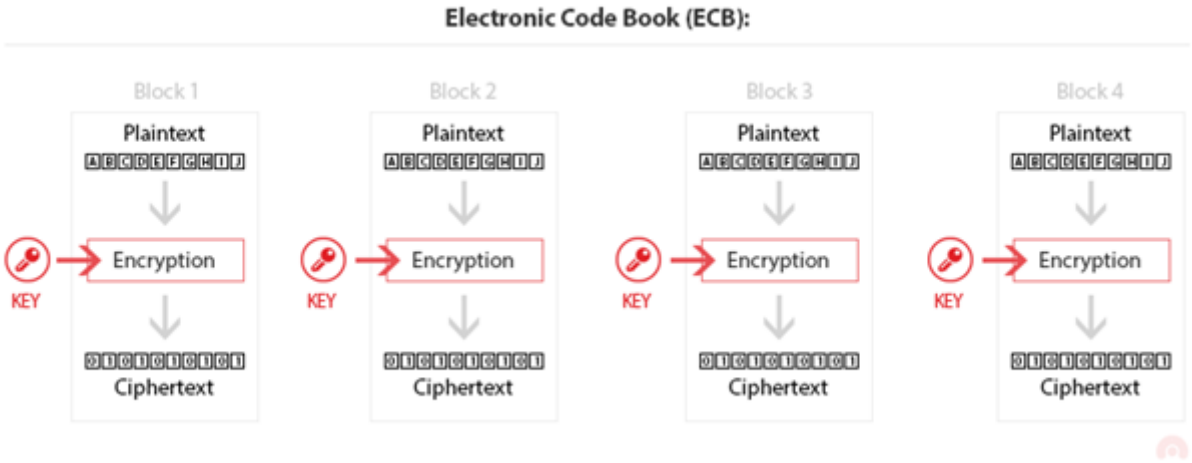
Voici un exemple d'opération de décryptage XOR.

Ciphertext		00011011 01110110 00001111 00011110 01111100 01110101
Key	S3CR3T	01010011 00110011 01000011 01010010 00110011 01010100
Message	Hello!	01001000 01000101 01001100 01001100 01001111 00100001

Les algorithmes de chiffrement par blocs

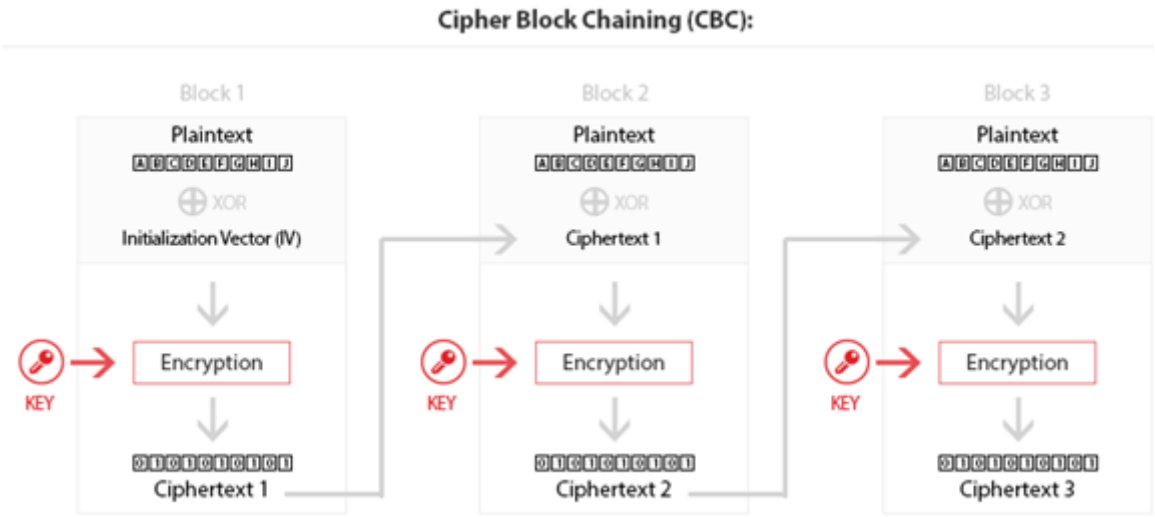
Livre de code électronique (ECB)

Chaque bloc de données est chiffré séparément et concaténé à la fin. Un inconvénient majeur de la BCE est que si le même bloc de données est chiffré, il générera toujours le même texte chiffré. Le traitement parallèle est possible car les blocs ne dépendent pas les uns des autres.



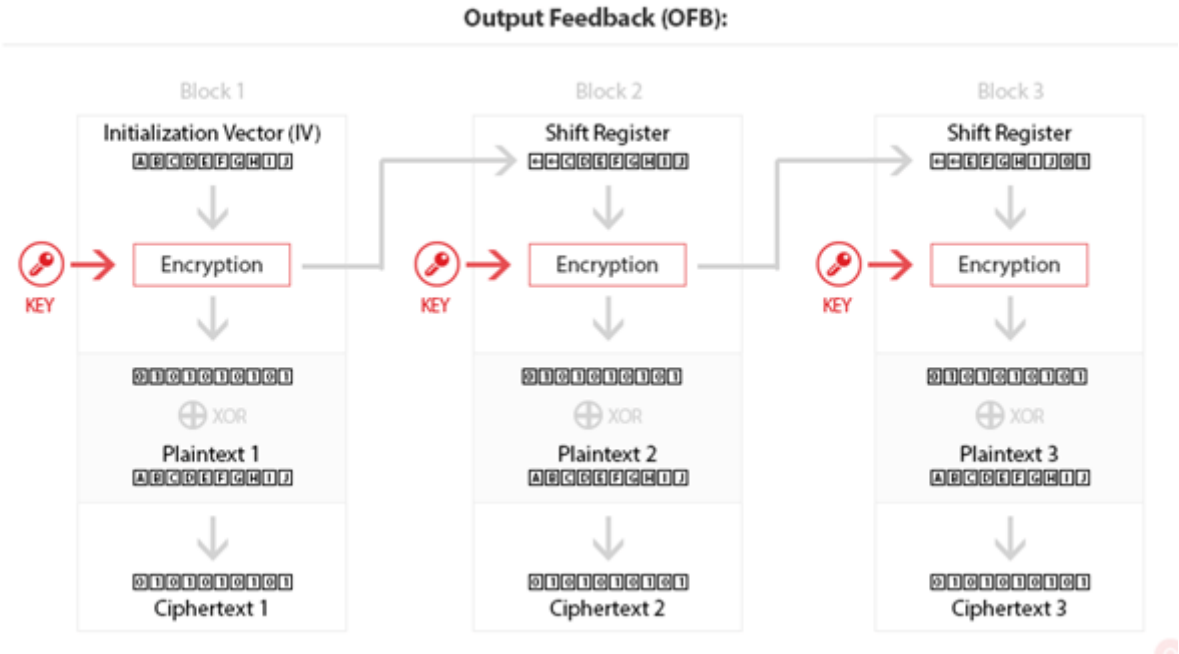
Chaînage de blocs de chiffrement (CBC)

Chaque bloc est XORed avec le texte chiffré précédent avant le chiffrement. Un vecteur d'initialisation est nécessaire pour le premier cryptage de bloc en clair. Le traitement parallèle n'est pas possible car les blocs sont chaînés.



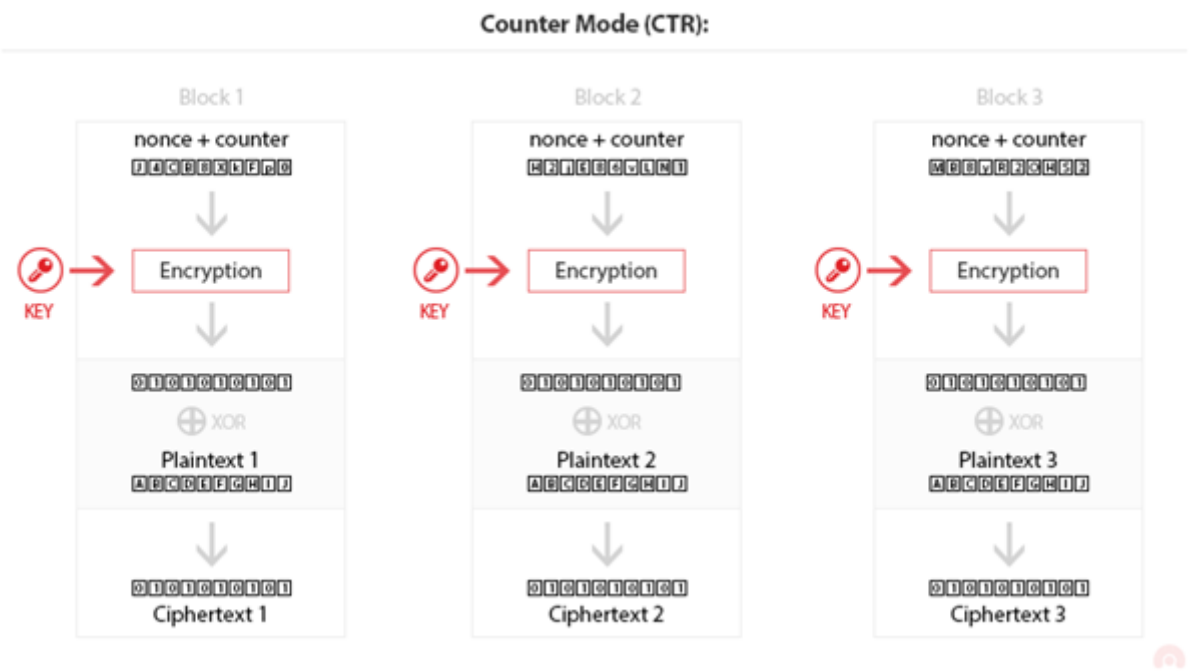
Boucle de retour de chiffrement (CFB Ciphers feedback)

Transforme un chiffrement par blocs en chiffrement de flux en sélectionnant un nombre de bits à XOR à chaque itération.



Mode compteur (CTR)

Chaque bloc possède un compteur de nonce et un compteur d'itération qui est d'abord chiffré, puis XOR avec un bloc de texte en clair. Puis le nonce change et le compteur s'incrémente à chaque itération.



Ciphers de flux

Les chiffrements de flux sont des chiffrements qui utilisent une méthode de chiffrement qui chiffre les données un bit ou un octet à la fois. Chaque bit est chiffré avec une clé différente. Alors que les chiffrements de flux ne sont pas beaucoup utilisés dans la cryptographie moderne, un exemple populaire de chiffrement de flux est le chiffrement RC4.

Code d'authentification du message (MAC)

Le code d'authentification de message (ou checksum cryptographique) est une méthode qui est utilisée pour vérifier l'authenticité ainsi que l'intégrité d'un message. Il accepte deux paramètres d'entrée, une clé secrète et un message de longueur arbitraire, et le résultat est appelé une étiquette.

Si la balise MAC de l'expéditeur et la balise MAC calculée du récepteur correspondent, cela signifie que le message n'a pas été falsifié. Si elles ne correspondent pas, cela signifie que le message a été modifié pendant la transmission.
Code d'authentification de message basé sur le hachage (HMAC)

HMAC est un type de MAC qui utilise une fonction de hachage. Il accepte deux paramètres d'entrée, une clé secrète et un message de longueur arbitraire.

Voici un exemple de fonction HMAC utilisant l'algorithme de hachage SHA256.

```
HMAC_SHA256("s3cr3tk3y","Hello World") = 2d9615ee921dab63c7c4c839842703fe338db46fdf17593a681bcee2c
```

L'illustration suivante montre le fonctionnement de la fonction HMAC.

CHAPITRE 3 : LES CERTIFICATS

Dans cette troisième partie de notre série, nous discuterons des trois différents types de certificats TLS / SSL, des trois formes de validation et de la manière de les générer.

Dans les parties 1 et 2 de cette série, nous avons définis le protocole, sa terminologie et ses bases. Dans ce troisième volet, nous décrirons les certificats TLS / SSL et leur utilisation.

Comme nous l'avons déjà vu, une connexion sécurisée peut être utilisée pour crypter des données et protéger nos données contre l'exposition à des tiers.

Pour que le chiffrement ait lieu, le serveur doit utiliser un certificat TLS / SSL. Un certificat TLS / SSL lie essentiellement une identité à une paire de clés qui sont ensuite utilisées par le serveur pour crypter et signer les données.

Autorité de certification (CA)

Une autorité de certification est une entité qui émet des certificats TLS / SSL ou numériques. Ces autorités ont leur propre certificat pour lequel elles utilisent leur clé privée pour signer le certificat TLS / SSL ou numérique délivré. Ce certificat est connu sous le nom de certificat racine.

Le certificat racine de l'autorité de certification, et donc la clé publique, est installé et approuvé par défaut dans les navigateurs tels que Chrome, Firefox et Edge. Ceci est nécessaire pour valider que le certificat d'un site Web visité a été signé par la clé privée de l'AC. Parmi les autorités CA les plus connues, on a Comodo, GlobalSign, DigiCert, GeoTrust, Thawte et Symantec.

Types de certificats TLS / SSL

Les certificats TLS / SSL sont disponibles dans différents types, regroupés par niveau de validation ou configuration de domaine.

Single-Domain (Domaine unique)

Ce type de certificat est utilisé pour sécuriser uniquement un nom d'hôte (ou un nom de domaine complet (FQDN) de nom de domaine complet) ou un sous-domaine. Par exemple, vous pouvez obtenir un certificat pour www.example.com ou my.example.com. Dans tous les

cas, mail.example.com ne sera pas sécurisé. Ni aucun autre sous-domaine. Le certificat ne sera valide que pour le nom d'hôte que vous spécifiez lors de l'enregistrement.

Wildcard (générique)

Ce type de certificat est utilisé pour sécuriser un domaine entier avec tous ses sous-domaines. Par exemple, * .example.com est utilisé dans l'enregistrement, ce qui signifie que tous les fichiers mail.example.com, secret.example.com, admin.example.com seront sécurisés (ainsi que tout autre sous-domaine). Gardez à l'esprit que chaque domaine peut être pointé vers un serveur différent. Le même certificat peut être utilisé sur plusieurs serveurs tant que le domaine est le même.

Multi-domaine

Ce type de certificat est utilisé pour sécuriser plusieurs noms de domaine différents.

Validation de certificat TLS / SSL

Validation de domaine

Il validera seulement que la personne qui demande un certificat est le propriétaire du nom de domaine (ou a au moins une sorte d'accès pour le faire). Ce type de validation ne prend généralement que quelques minutes, mais peut prendre jusqu'à quelques heures.

Validation de l'organisation

L'autorité de certification (CA) non seulement valide la propriété du domaine mais aussi l'identité du propriétaire. Cela signifie qu'un propriétaire peut être invité à fournir des documents d'identification personnels qui prouvent son identité. La validation peut prendre plusieurs jours avant que le certificat soit délivré.

Validation étendue (EV)

C'est le plus haut niveau de validation et il inclut la validation de la propriété du domaine, l'identité du propriétaire, ainsi que la preuve d'enregistrement légal d'une entreprise.

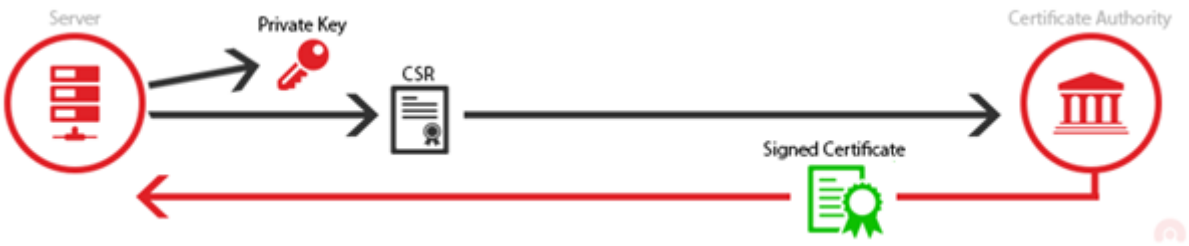
Génération de certificat

Pour qu'une autorité de certification émette un certificat, elle doit d'abord avoir la RSC de notre serveur, qui signifie Demande de signature de certificat (Certificat Signing Request). Nous créons d'abord une clé privée qui sera utilisée pour décrypter notre certificat, puis nous générons un CSR.

Lors de la création du CSR, il nous sera demandé de spécifier le nom de domaine ainsi que des détails sur notre organisation tels que le nom, le pays et le courrier électronique. L'exemple suivant montre un CSR:

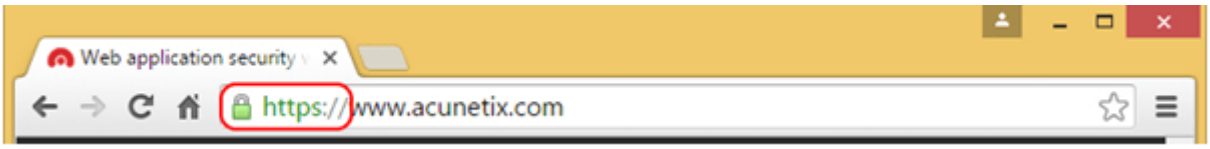
```
-----BEGIN CERTIFICATE REQUEST-----
MIICtzCCAZ8CAQAwUTELMAkGA1UEBhMCQ1kxCjAIBgNVBAgMAWExCzAJBgNVBAcM
Ak1lMQowCAYDVQQKDAFhMQowCAYDVQQLDAFhMREwDwYDVQQDDAh0ZXN0LmNvbTCCASIwDQYJKoZIhvcNAQEBBQADggEPADCCAQ
AT9KAZ3w+2uwjrhJG5w52psENUT+tTOM1DgIV1KSbZcvD5bS/X7RvfsOS3Q/y9wG
Q53rbZqK19y5CtM808wGOQhrNfp3M1EA6+m7RRO1Yw2Rp+wLY0aA9UzjzImaL5Sr
/job1EoJWzKClY20GXB6HKn+wJ/n4sz725bF616r4yoBY1f4gYBn3QW+sQXLrDsC
AwEAAaAhMB8GCSqGSIb3DQEJBzESDBBkZmpoZ2tqaGpoZGZramhnMA0GCSqGSIb3
DQEBCwUAA4IBAQCFQ7/R+/ioSj7X4gs+GBbDHEcnJHshwoX9vVBDYvOoQ56iER7f
cEtja18yeXu3PNyeOoDLSYd0FhM16XKL1J011Iy46Vb8RMdS4JNEx2yob3W/bIAS
z2n+p58zp2/Gp7gG2LtH+RwcvRGRGdKhrsU8D+fHGHpSGvGJg++IhS2HZvTs5pkD
3AwMpDojVYtWuJteDVHGc4PH5TeJot7+6lGetkhq1uRhD4KjJDY5KUvCNQIjeoaL
eFf/xGp6JGNE5taK2liBs+QlgLZc8Sm7fTYCi0YXAsEhMm9HjbXX28Ou6zTroDP3
elT3tN9pd1aG6ujGjkrM6T8JiVWxqYFEnFqd
-----END CERTIFICATE REQUEST-----
```


Le CSR est ensuite soumis à l'autorité de certification afin de créer notre certificat. Lorsque le certificat est prêt, il sera envoyé à notre email au format * .crt et il devra ensuite être installé sur le serveur.



Navigation sécurisée

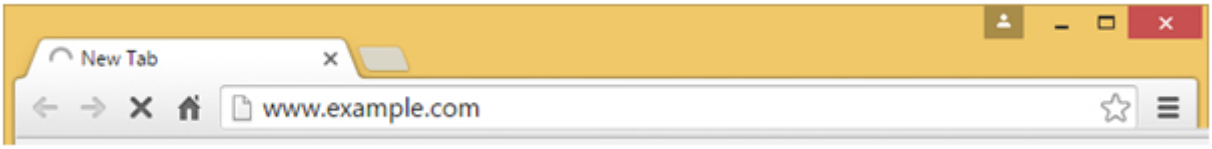
Identifier si un site Web a un certificat valide et que vous êtes sur une connexion sécurisée est très facile. Tout ce que vous avez à faire est de regarder l'état de la barre d'URL en haut à gauche de notre écran (il est similaire dans tous les principaux navigateurs).



Le verrou vert avec le protocole https: // indique que la connexion au serveur Web est cryptée et sécurisée.

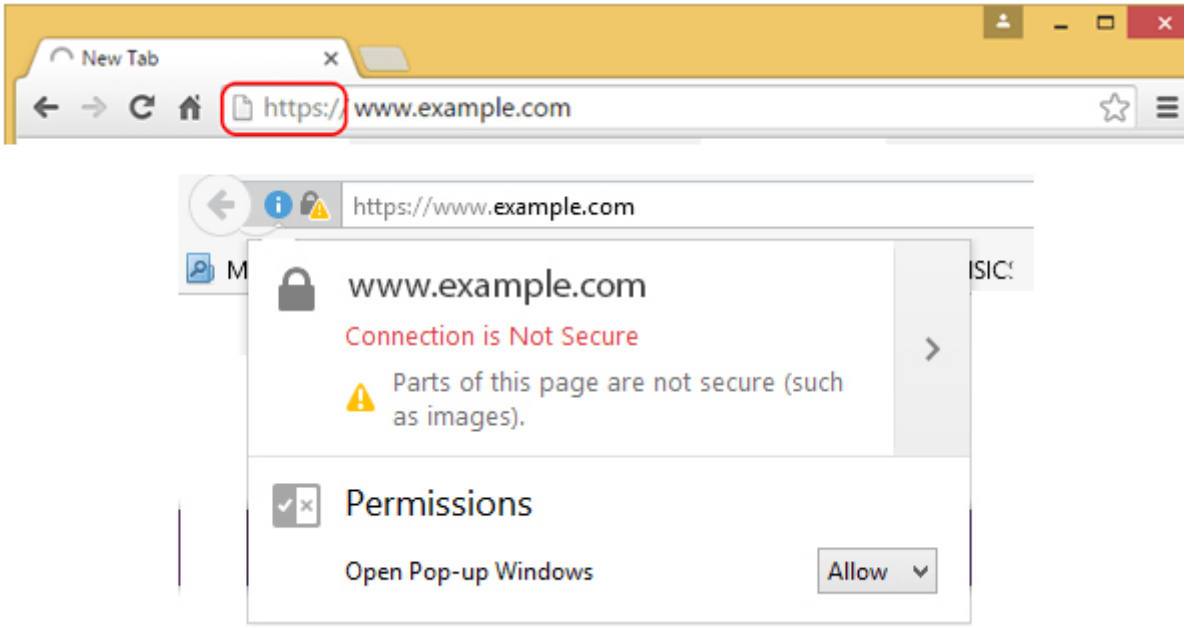
Navigation non sécurisée

Identifier un site Web non sécurisé est tout aussi facile. Il n'y a pas de verrou vert et il n'y a aucune mention de HTTPS.



Toutefois, l'identification d'un site Web non sécurisé utilisant le protocole HTTPS peut être difficile si vous êtes un utilisateur non expérimenté. La raison en est que même si le site Web utilise un certificat SSL / TLS et que HTTPS est utilisé, certaines parties du contenu sont transmises via HTTP.

Dans ce cas, en fonction du navigateur, nous verrons l'un des éléments suivants :



C'est ce que nous appelons le contenu mixte. Les vulnérabilités de contenu mixte vont à l'encontre de l'objectif d'une connexion sécurisée, d'autant plus que lorsque vous demandez des fichiers à partir d'un site Web auquel nous sommes connectés, le navigateur envoie automatiquement nos cookies d'authentification le long de la requête.

Par conséquent, si une ressource, telle qu'une image, est chargée à l'aide de HTTP, notre requête envoyée via HTTP n'est pas cryptée; ce qui signifie qu'un hacker reniflant le réseau pourra voir cette requête en clair. Cela compromet la sécurité d'une session « sécurisée », car un hacker peut désormais utiliser le cookie pour se connecter au site Web et usurper l'identité d'une victime.

ANNEXE A : SSL vs TLS QUELLE EST LA DIFFERENCE ?

La sécurité sur Internet est un peu comme la soupe à l'alphabet – SSL, TLS, ECC, SHA, la liste continue. Tous ces acronymes peuvent rendre difficile la compréhension de ce dont vous avez réellement besoin. L'intérrogation qui revient le plus souvent : quelle est la différence entre SSL (Secure Socket Layers) et TLS (Transport Layer Security)? Vous savez que vous voulez sécuriser votre site Web (ou tout autre type de communication), mais avez-vous besoin de SSL ? TLS ? Tous les deux ? Je vais essayer comme d'habitude d'apporter un début de réponse

Une brève histoire sur le SSL et TLS

SSL et TLS sont des protocoles cryptographiques qui fournissent une authentification et un cryptage de données entre des serveurs, des machines et des applications fonctionnant sur un réseau (par exemple un client se connectant à un serveur Web). SSL est le prédécesseur de TLS. Au fil des années, de nouvelles versions des protocoles ont été publiées pour corriger les vulnérabilités et prendre en charge des suites de chiffrement et des algorithmes plus solides et plus sécurisés.

SSL a été développé à l'origine par Netscape et est arrivé sur la scène en 1995 avec SSL 2.0 (1.0 n'a jamais été publié). La version 2.0 a été rapidement remplacée par SSL 3.0 en 1996 après qu'un certain nombre de vulnérabilités aient été trouvées. Remarque : Les versions 2.0 et 3.0 sont parfois écrites en tant que SSLv2 et SSLv3.

TLS a été introduit en 1999 comme une nouvelle version de SSL et était basé sur SSL 3.0:

Les différences entre ce protocole et SSL 3.0 ne sont pas dramatiques, mais elles sont suffisamment importantes pour que TLS 1.0 et SSL 3.0 n'interopèrent pas. Source

Devriez-vous utiliser le SSL ou TLS ?

Les deux SSL 2.0 et 3.0 ont été déconseillés par l'IETF (en 2011 et 2015, respectivement). Au cours des années, des vulnérabilités ont été et continuent d'être découvertes dans les protocoles SSL obsolètes (par exemple POODLE, DROWN). La plupart des navigateurs modernes afficheront une expérience utilisateur dégradée (par exemple une ligne via le

cadenas ou https dans la barre d'URL, des avertissements de sécurité) lorsqu'ils rencontrent un serveur Web utilisant les anciens protocoles. Pour ces raisons, vous devez désactiver SSL 2.0 et 3.0 dans la configuration de votre serveur, en laissant uniquement les protocoles TLS activés.

Les certificats ne dépendent pas des protocoles

Avant que quiconque ne s'inquiète de devoir remplacer ses certificats SSL existants par des certificats TLS, il est important de noter que les certificats ne dépendent pas des protocoles. Autrement dit, vous n'avez pas besoin d'utiliser un certificat TLS par rapport à un certificat SSL. Alors que de nombreux fournisseurs ont tendance à utiliser l'expression «certificat SSL / TLS», il est peut-être plus exact de les appeler «Certificats à utiliser avec SSL et TLS», car les protocoles sont déterminés par la configuration de votre serveur.

Il est probable que vous continuerez à voir des certificats appelés certificats SSL, car à ce stade, c'est le terme que plus de gens connaissent, mais nous commençons à voir une utilisation accrue du terme TLS dans l'industrie. SSL / TLS est un compromis commun jusqu'à ce que plus de gens se familiarisent avec TLS.

SSL et TLS sont-ils différents sur le plan cryptographique ?

En vérité, la réponse à cette question est oui, mais vous pouvez en dire autant des versions historiques de SSL 2 et 3 ou des versions 1 de TLS avec 1.1, 1.2 ou 1.3. SSL et TLS sont tous deux sur le même protocole mais en raison des différences de version, SSL 2 n'était pas interopérable avec la version 3, et SSL version 3 pas avec TLS version 1. Vous pourriez argumenter que Transport Layer Security (TLS) était juste un nouveau nom pour SSL v4 – essentiellement, nous parlons du même protocole.

Chaque nouvelle version du protocole vient et vient avec ses propres améliorations et / ou de nouvelles fonctionnalités / obsolètes. SSL version 1 n'a jamais été publié, la version 2 a eu quelques défauts majeurs, SSL version 3 était une réécriture de la version 2 (pour corriger ces failles) et TLS version 1 une amélioration de SSL version 3. Depuis la sortie de TLS 1.0 les changements ont été moins significatifs, mais jamais moins importants.

Il convient de noter ici que SSL et TLS se réfèrent simplement à la poignée de main qui a lieu entre un client et un serveur. La poignée de main ne fait aucun cryptage en soi, elle accepte simplement un secret partagé et le type de cryptage qui va être utilisé.

ANNEXE B : COMMENT UTILISER UN SERVEICE REST AVEC UN CERTIFICATI SSL ?

Dans nos environnements de développement, nous avons configuré des certificats SSL auto-signés pour nos applications. Évidemment, je ne dépenserai pas d'argent pour des serveurs de développement internes. Donc, j'ai opté pour des certificats auto-signés. Et c'est là où les problèmes commencent.

J'utilise le client du repos Jersey pour appeler des services REST.

```
TrustManager[] trustAllCerts = new TrustManager[] { new X509TrustManager() {
public X509Certificate[] getAcceptedIssuers() {
return null;
}
public void checkClientTrusted(X509Certificate[] certs, String authType) {}
public void checkServerTrusted(X509Certificate[] certs, String authType) {}
} };

SSLContext context = SSLContext.getInstance("TLS");
context.init(null, trustAllCerts, new SecureRandom());
HttpsURLConnection.setDefaultSSLSocketFactory(context.getSocketFactory());

ClientConfig config = new DefaultClientConfig();
config.getProperties().put(HTTPSProperties.PROPERTY_HTTPS_PROPERTIES, new HTTPSProperties(new Hostname
@Override
public boolean verify(String s, SSLSession sslSession) {
return true;
}
}, context));

Client client = Client.create(config);
client.setFollowRedirects(true);
WebResource resource = client.resource("https://myserver/myws");
resource.accept(MediaType.APPLICATION_JSON_TYPE);
String result = resource.post(String.class);
```